

Chapter 10: Constraints

Algorithms for Optimization, 2nd Ed.
Kochenderfer & Wheeler (2026)

Lecture Slides

Chapter Outline

- 1 Constrained Optimization
- 2 Constraint Types
- 3 Transformations to Remove Constraints
- 4 Removing Affine Equality Constraints
- 5 Projected Iterative Methods
- 6 Lagrange Multipliers & KKT Conditions
- 7 Inequality Constraints & KKT Conditions
- 8 Slack Variables
- 9 Penalty Methods
- 10 Method of Multipliers (Augmented Lagrangian)
- 11 Interior Point Methods
- 12 Comparison of Methods
- 13 Summary & Exercises

Constrained Optimization — Motivation

Previous chapters assumed:

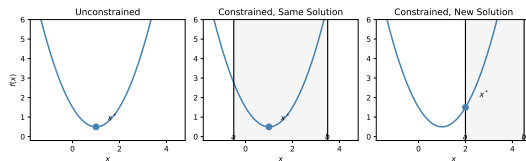
$$x \in \mathbb{R}^n \quad (\text{unconstrained})$$

Reality: design variables are often restricted.

Engineering examples

- Aircraft: structural load limits ($\sigma \leq \sigma_{\max}$)
- Portfolio: budget constraint ($\sum w_i = 1$)
- Robot: joint angle bounds ($\theta_{\min} \leq \theta \leq \theta_{\max}$)

Constraints **restrict** the feasible set $\mathcal{X} \subset \mathbb{R}^n$.
They *may or may not* change the optimal solution.



Constraints can change the solution (right)
or leave it unchanged (middle).

General Constrained Optimization Problem

Standard Form (Eq. 10.2)

$$\begin{array}{ll} \underset{x}{\text{minimize}} & f(x) \\ \text{subject to} & h_i(x) = 0, \quad i = 1, \dots, \ell \\ & g_j(x) \leq 0, \quad j = 1, \dots, m \end{array}$$

- $f : \mathbb{R}^n \rightarrow \mathbb{R}$ — **objective function**
- $h_i : \mathbb{R}^n \rightarrow \mathbb{R}$ — **equality constraints**
- $g_j : \mathbb{R}^n \rightarrow \mathbb{R}$ — **inequality constraints** (≤ 0)
- Set-membership $x \in \mathcal{X}$ can be cast as $h(x) = (x \notin \mathcal{X})$

Greater-than conversion

$$g(x) \geq 0 \iff -g(x) \leq 0$$

Numerical example

$$\begin{array}{ll} \text{Minimize} & x_1^2 + x_2^2 \\ \text{s.t.} & x_1 + x_2 = 1 \text{ (equality)} \\ & x_1 \geq 0, x_2 \geq 0 \text{ (inequality)} \end{array}$$

Constraint Types

1. Equality constraints

$$h(x) = 0$$

Restricts x to a **surface** (manifold) of dimension $n - \ell$.

2. Inequality constraints

$$g(x) \leq 0$$

Restricts x to a **half-space** or curved feasible region.

3. Bound constraints (special case)

$$a \leq x_i \leq b$$

Can be written as $g_1(x_i) = x_i - b \leq 0$
and $g_2(x_i) = a - x_i \leq 0$.

Type	Form
Equality	$h(x) = 0$
Inequality	$g(x) \leq 0$
Bound	$a \leq x_i \leq b$
Box	$x \in [a, b]$
Set-membership	$x \in \mathcal{X}$

Key insight

Any constraint type can be converted to equality or inequality form — giving a *unified* framework.

Removing Bound Constraints via Variable Transformation

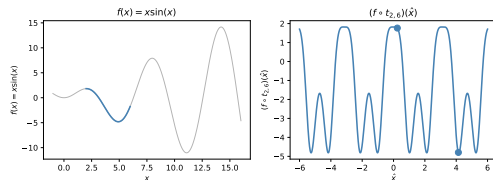
For $a \leq x \leq b$, define the **sigmoid transform**:

$$x = t_{a,b}(\hat{x}) = a + (b - a) \frac{2\hat{x}}{1 + \hat{x}^2} \quad (\text{sinusoidal variant})$$

More commonly:

$$t_{a,b}(\hat{x}) = a + (b - a) \cdot \sin^2(\hat{x})$$

- $\hat{x} \in \mathbb{R}$ is **unconstrained**
- $t_{a,b}$ maps $\mathbb{R} \rightarrow (a, b)$
- Substitute into f to get an unconstrained problem
- Apply *any* unconstrained solver



Example 10.1: $\min x \sin(x)$, $x \in [2, 6]$
Right: two minima at $\hat{x} \approx 0.242$ and 4.139 ,
both map to $x \approx 4.914$.

Python: Bound-Constrained Minimization via Transform

```
import numpy as np
from scipy.optimize import minimize

a, b = 2.0, 6.0

def t(xhat):
    """Sigmoid transform: maps R -> (a, b)."""
    return a + (b - a) * np.sin(xhat)**2

def f_transformed(xhat):
    x = t(xhat)
    return x * np.sin(x)

# Solve unconstrained problem
results = []
for x0 in np.linspace(-6, 6, 20):
    res = minimize(f_transformed, x0, method='L-BFGS-B')
    results.append((res.fun, t(res.x[0])))

# Best solution in original space
best = min(results, key=lambda r: r[0])
print(f"f* = {best[0]:.4f}, x* = {best[1]:.4f}")
# Output: f* = -4.8141, x* = 4.9136
```

Listing 1: Example 10.1 in Python — bound-constrained minimization

Removing Affine Equality Constraints

Setting: $Ax = b$ with $A \in \mathbb{R}^{m \times n}$, $\text{rank}(A) = m < n$.

LQ decomposition of A :

Eq. 10.6

$$A = \begin{bmatrix} L & 0 \end{bmatrix} Q$$

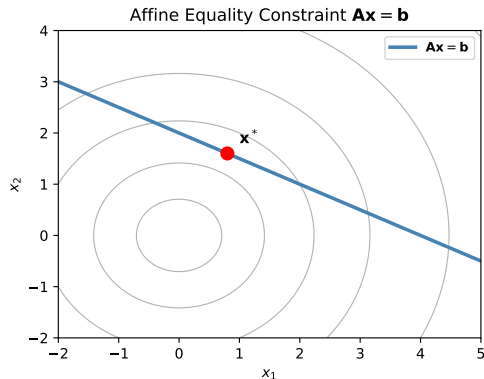
$L \in \mathbb{R}^{m \times m}$ lower-triangular, $Q \in \mathbb{R}^{n \times n}$ orthogonal.

Apply change of variables $Qx = y$:

$$y = \begin{bmatrix} y_{1:m} \\ y_{(m+1):n} \end{bmatrix}$$

Constraint satisfied when $y_{1:m}^* = L^{-1}b$.

Free variables $y_{(m+1):n}$ can be optimized without constraint.



Affine constraint $x_1 + 2x_2 = 4$ restricts optimum to the line; constrained $x^* = (4/5, 8/5)$.

Example 10.2

Constraint $h(x) = c_1x_1 + \dots + c_nx_n = 0$: 8 / 31

Python: Affine Equality Constraint via Substitution

```
import numpy as np
from scipy.optimize import minimize

# Constraint:  $x_1 + 2x_2 + 3x_3 = 6 \Rightarrow x_1 = 6 - 2x_2 - 3x_3$ 
def f_reduced(y):
    """y = [x2, x3]; x1 is determined by the constraint."""
    x2, x3 = y
    x1 = 6.0 - 2*x2 - 3*x3
    return x2**2 + x3 - (2*x2 + 3*x3 - 6)**3

x0 = [1.0, 1.0]
res = minimize(f_reduced, x0, method='L-BFGS-B')
x2_opt, x3_opt = res.x
x1_opt = 6.0 - 2*x2_opt - 3*x3_opt

print(f"x* = ({x1_opt:.4f}, {x2_opt:.4f}, {x3_opt:.4f})")
print(f"Constraint satisfied: {np.isclose(x1_opt + 2*x2_opt + 3*x3_opt, 6)}")
```

Listing 2: Minimize $x_1^3 + x_2^2 + x_3$ subject to $x_1 + 2x_2 + 3x_3 = 6$

Projected Gradient Descent

Idea: Run unconstrained descent, then **project** back to the feasible set.

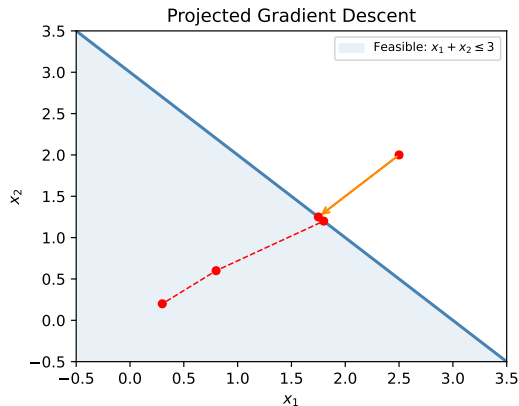
Algorithm 10.1 — Projected Descent

- 1 Start at feasible $x^{(1)}$
- 2 Compute gradient step: $\tilde{x} \leftarrow x^{(k)} - \alpha \nabla f(x^{(k)})$
- 3 **Project:** $x^{(k+1)} \leftarrow \arg \min_{x \in \mathcal{X}} \|x - \tilde{x}\|$
- 4 Repeat until convergence

Projection onto a convex set $\mathcal{X}$:

$$\Pi_{\mathcal{X}}(\tilde{x}) = \arg \min_{x \in \mathcal{X}} \|x - \tilde{x}\|_2$$

- Box $[a, b]$: componentwise clip
- Halfspace $\{x : c^\top x \leq d\}$: closed-form shift
- General convex $\mathcal{X}$: solve a QP



Orange arrows: unconstrained gradient steps.
Red path: projected iterates walking along the boundary.

Python: Projected Gradient Descent

```
import numpy as np

def project_halfspace(x, a, b_val):
    """Project x onto {x : a^T x <= b_val}."""
    violation = a @ x - b_val
    if violation > 0:
        x = x - violation * a / (a @ a)
    return x

def f(x):      return (x[0]-3)**2 + (x[1]-3)**2
def grad_f(x): return np.array([2*(x[0]-3), 2*(x[1]-3)])

a = np.array([1.0, 1.0])
b_val = 3.0
x = np.array([2.5, 2.0])    # infeasible start
alpha = 0.1

for _ in range(50):
    x_new = x - alpha * grad_f(x)
    x = project_halfspace(x_new, a, b_val)

print(f"x* = {x},    constraint satisfied: {a @ x <= b_val + 1e-8}")
# x* ~= [1.5, 1.5],    constraint satisfied: True
```

Listing 3: Projected gradient descent onto a half-space $x_1 + x_2 \leq 3$

Lagrange Multipliers — Equality Constraints

Optimality condition: at a constrained minimum x^* , the gradient of f is *perpendicular* to the constraint surface, i.e. parallel to ∇h :

Eq. 10.15

$$\nabla f(x) = \lambda \nabla h(x)$$

$\lambda \in \mathbb{R}$ is the **Lagrange multiplier**.

Define the **Lagrangian**:

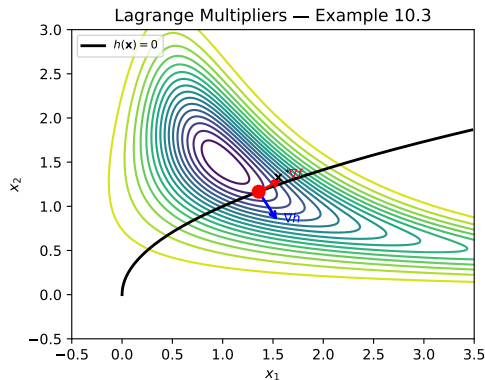
Eq. 10.16

$$\mathcal{L}(x, \lambda) = f(x) - \lambda h(x)$$

Setting $\nabla \mathcal{L} = 0$ gives both conditions simultaneously:

$$\nabla_x \mathcal{L} = 0 \Rightarrow \nabla f = \lambda \nabla h$$

$$\nabla_\lambda \mathcal{L} = 0 \Rightarrow h(x) = 0$$



Multiple Equality Constraints

Generalized Lagrangian — Multiple Equalities

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) - \sum_{i=1}^{\ell} \lambda_i h_i(\mathbf{x})$$

Setting $\nabla \mathcal{L} = 0$ requires all $\ell + 1$ conditions:

$$\nabla f(\mathbf{x}) = \sum_{i=1}^{\ell} \lambda_i \nabla h_i(\mathbf{x}), \quad h_i(\mathbf{x}) = 0 \quad \forall i$$

Example 10.4 — Numerical Solution

Problem: $\min -\exp(-(x_1 x_2 - \frac{3}{2})^2 - (x_2 - \frac{3}{2})^2)$ subject to $x_1 - x_2^2 = 0$.

Lagrangian: $\mathcal{L} = -\exp(\cdots) - \lambda(x_1 - x_2^2)$

Setting partial derivatives to zero and solving yields:

$$\mathbf{x}^* \approx [1.358, 1.165], \quad \lambda^* \approx 0.170$$

Python: Method of Lagrange Multipliers

```
import numpy as np
from scipy.optimize import fsolve

def f(x):
    return -np.exp(-((x[0]*x[1] - 1.5)**2 + (x[1] - 1.5)**2))

def grad_L(vars):
    x1, x2, lam = vars
    fval = f([x1, x2])
    # dL/dx1 = 2*x2*f*(x1*x2 - 3/2) - lam
    dLdx1 = 2*x2*fval*(x1*x2 - 1.5) - lam
    # dL/dx2 = f*(2*lam*x2 - 2*x1*(x1*x2-3/2) - 2*(x2-3/2))
    dLdx2 = 2*lam*x2*fval + fval*(-2*x1*(x1*x2-1.5) - 2*(x2-1.5))
    # dL/dlambda = -(x2^2 - x1) (constraint)
    dLdlam = -(x2**2 - x1)
    return [dLdx1, dLdx2, dLdlam]

x0 = [1.0, 1.0, 0.1]
sol = fsolve(grad_L, x0, full_output=True)
x1s, x2s, lams = sol[0]
print(f"x* = [{x1s:.4f}], {x2s:.4f}], lambda* = {lams:.4f}")
# x* = [1.3580, 1.1650], lambda* = 0.1700
```

Listing 4: Solving the system $\nabla \mathcal{L} = 0$ numerically (Example 10.4)

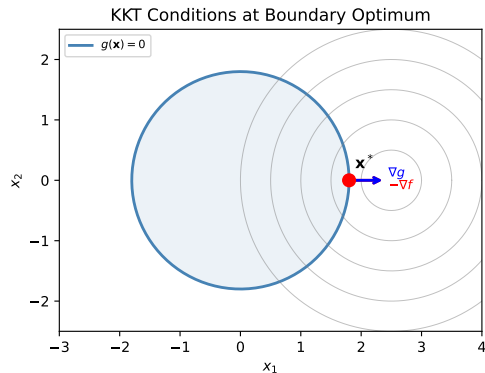
KKT Conditions for Inequality Constraints

Extend to $g_j(\mathbf{x}) \leq 0$ using the **Generalized Lagrangian**:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\mathbf{x}) - \sum_i \lambda_i h_i(\mathbf{x}) - \sum_j \mu_j g_j(\mathbf{x})$$

KKT Necessary Conditions

- 1 **Stationarity:** $\nabla_{\mathbf{x}} \mathcal{L} = 0$
- 2 **Primal feasibility:** $h_i(\mathbf{x}^*) = 0, \quad g_j(\mathbf{x}^*) \leq 0$
- 3 **Dual feasibility:** $\mu_j \geq 0$
- 4 **Complementary slackness:**
 $\mu_j g_j(\mathbf{x}^*) = 0$ for all j



At a boundary optimum $\mathbf{x}^*$:
 $-\nabla f$ and ∇g must be aligned ($\mu > 0$).
Interior optimum: $\mu = 0$ (constraint inactive).

Complementary slackness

Either $\mu_i = 0$ (inactive constraint)

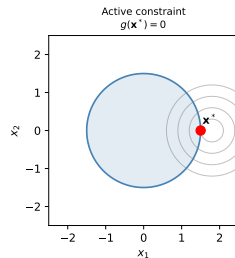
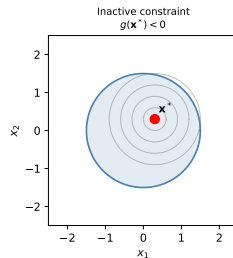
Active vs. Inactive Constraints

Inactive constraint ($g(\mathbf{x}^*) < 0$)

- $\mathbf{x}^*$ is in the *interior* of feasible region
- KKT: $\mu_j = 0$ (constraint has no effect)
- Effectively an unconstrained problem locally

Active constraint ($g(\mathbf{x}^*) = 0$)

- $\mathbf{x}^*$ lies on the *boundary*
- KKT: $\mu_j > 0$ (constraint is binding)
- Gradient of f has a component into infeasible region



Left: optimum in interior (inactive).
Right: optimum on boundary (active).

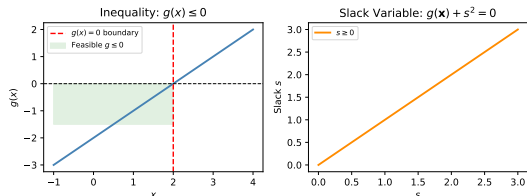
Slack Variables

Convert inequality $g(x) \leq 0$ to equality by adding a **slack variable** $s \geq 0$:

$$g(x) \leq 0 \iff g(x) + s = 0, \quad s \geq 0$$

Use $s = t^2$ to enforce $s \geq 0$ automatically:

$$g(x) + t^2 = 0 \quad (t \in \mathbb{R}, \text{ unconstrained})$$



Slack $s = t^2 \geq 0$ converts the inequality into an equality.

Transformed Problem

$$\underset{x,t}{\text{minimize}} \quad f(x)$$

$$\text{s.t.} \quad g(x) + t^2 = 0$$

Now an **equality**-constrained problem.

Generalizes: for m inequalities introduce $t_1, \dots, t_m$.

Numerical example

$$g(x) = x - 5 \leq 0$$

$$\text{Add } s = t^2: x - 5 + t^2 = 0$$

$$\Rightarrow x = 5 - t^2 \leq 5 \checkmark$$

Penalty Methods — Overview

Idea: add a penalty term $\rho p(x)$ to the objective that is zero when feasible, *positive* when infeasible:

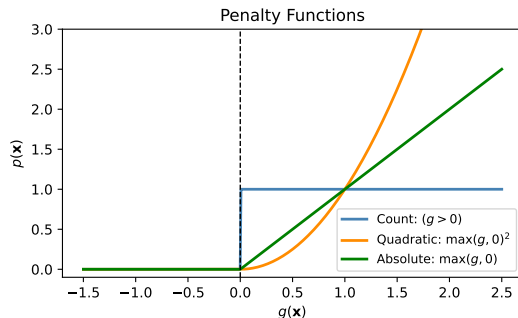
$$\underset{x}{\text{minimize}} \quad f(x) + \rho p(x)$$

Common penalty functions

- **Count:** $p(x) = \sum_j (g_j(x) > 0)$
- **Quadratic:** $p(x) = \sum_j \max(g_j(x), 0)^2$
- **For equality:** $p(x) = \sum_i h_i(x)^2$

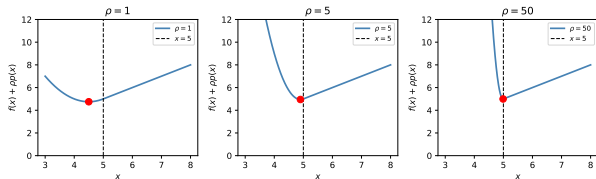
Strategy: solve a sequence with increasing ρ :

$$\rho_1 < \rho_2 < \dots \rightarrow \infty$$



Different penalty functions as a function of constraint value $g(x)$.

Penalty Method — Convergence and Limitation



Penalized objective for $\min x, x \geq 5$.
Minimum $x^* = 5 - \frac{1}{2\rho}$ approaches 5 as $\rho \rightarrow \infty$.

Key limitation (Example 10.5)

For $\min x$ subject to $x \geq 5$:
penalized unconstrained minimum is

$$x^* = 5 - \frac{1}{2\rho}$$

which only *approaches* 5 — never equals it!

Requires $\rho \rightarrow \infty$ for feasibility.

Large ρ issues

- Ill-conditioned Hessian
- Slow convergence
- Numerical instability

Python: Quadratic Penalty Method

```
import numpy as np
from scipy.optimize import minimize

def quadratic_penalty_method(f, h, x0, k_max=20, rho=1.0, gamma=2.0):
    """
    f      : objective function
    h      : equality constraint function (vector-valued)
    x0     : initial point
    k_max  : number of penalty iterations
    rho    : initial penalty parameter
    gamma  : penalty growth factor (> 1)
    """
    x = np.array(x0, dtype=float)
    for k in range(k_max):
        def penalized(xv):
            hv = h(xv)
            return f(xv) + (rho / 2.0) * np.sum(hv**2)
        res = minimize(penalized, x, method='L-BFGS-B')
        x = res.x
        rho *= gamma
    return x

# Example: min x1^2+x2^2 s.t. x1+x2=1
f = lambda x: x[0]**2 + x[1]**2
h = lambda x: np.array([x[0] + x[1] - 1.0])
x_opt = quadratic_penalty_method(f, h, [0.5, 0.5])
print(f"x* = {x_opt}")    # x* ~= [0.5, 0.5]
```

Method of Multipliers — Augmented Lagrangian

Combines **Lagrange multipliers** + **quadratic penalty**:

Augmented Lagrangian (Eq. 10.37)

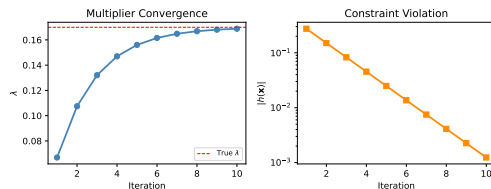
$$\mathcal{L}_\rho(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \frac{\rho}{2} \|\mathbf{h}(\mathbf{x})\|^2 + \boldsymbol{\lambda}^\top \mathbf{h}(\mathbf{x})$$

Algorithm 10.3 — Method of Multipliers

- 1 Initialize $\boldsymbol{\lambda} \leftarrow 0$, $\rho > 0$
- 2 **Inner minimize:**
 $\mathbf{x} \leftarrow \arg \min_{\mathbf{x}} [f(\mathbf{x}) + \frac{\rho}{2} \|\mathbf{h}(\mathbf{x})\|^2 + \boldsymbol{\lambda}^\top \mathbf{h}(\mathbf{x})]$
- 3 **Multplier update:**
 $\boldsymbol{\lambda} \leftarrow \boldsymbol{\lambda} + \rho \mathbf{h}(\mathbf{x})$
- 4 Increase $\rho \leftarrow \gamma \rho$
- 5 Repeat until $\|\mathbf{h}(\mathbf{x})\| \leq \varepsilon$

Advantage over pure penalty:

- Can achieve *exact feasibility* at finite ρ
- Multiplier update drives constraint to zero
- Better conditioned sub-problems



Left: $\lambda^{(k)} \rightarrow \lambda^*$. Right: constraint violation $\rightarrow 0$.

Python: Method of Multipliers (Augmented Lagrangian)

```
import numpy as np
from scipy.optimize import minimize

def method_of_multipliers(f, h, x0, k_max=20, rho=1.0, gamma=2.0):
    """
    Augmented Lagrangian method for equality constraints  $h(x) = 0$ .
    """
    x = np.array(x0, dtype=float)
    lam = np.zeros(len(h(x)))          # Lagrange multipliers
    for k in range(k_max):
        def aug_lagrangian(xv):
            hv = h(xv)
            return f(xv) + (rho / 2.0) * np.dot(hv, hv) + np.dot(lam, hv)
        res = minimize(aug_lagrangian, x, method='L-BFGS-B')
        x = res.x
        lam = lam + rho * h(x)          # multiplier update
        rho *= gamma
        if np.linalg.norm(h(x)) < 1e-8:
            break
    return x, lam

# Example: min  $x_1^2 + x_2^2$  s.t.  $x_1 + x_2 = 1$ 
f = lambda x: x[0]**2 + x[1]**2
h = lambda x: np.array([x[0] + x[1] - 1.0])
x_opt, lam_opt = method_of_multipliers(f, h, [2.0, 0.0])
print(f"x* = {x_opt}, lambda* = {lam_opt}")
# x* ~= [0.5, 0.5], lambda* ~= [-1.0]
```

Interior Point Methods — Barrier Functions

Idea: instead of penalizing infeasibility, add a **barrier** that $\rightarrow +\infty$ as x approaches the boundary from *inside*:

Log Barrier for $g_j(x) \leq 0$

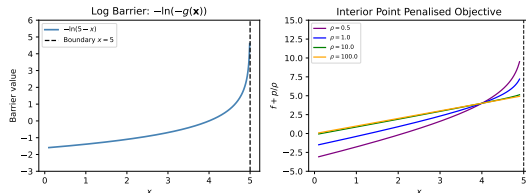
$$p(x) = - \sum_j \ln(-g_j(x))$$

Defined only when $g_j(x) < 0$ (interior).

Penalized Objective

$$\underset{x}{\text{minimize}} \quad f(x) + \frac{1}{\rho} p(x)$$

Start with small ρ , increase gradually.
The barrier shrinks, pushing solution to boundary.



Other barrier functions

- **Inverse:** $p(x) = - \sum_j \frac{1}{g_j(x)}$
- Stays strictly in interior
- Requires strictly feasible $x^{(0)}$

Interior Point Method — Algorithm and Properties

Algorithm 10.4 — Interior Point Method

❶ Start from **strictly feasible** $x^{(0)}$
(all $g_j(x^{(0)}) < 0$)

❷ Set $\delta \leftarrow \infty$

❸ **While** $\delta > \varepsilon$:

❶ $x' \leftarrow \arg \min_x [f(x) + p(x)/\rho]$

❷ $\delta \leftarrow \|x' - x\|$

❸ $x \leftarrow x'$

❹ $\rho \leftarrow \gamma \rho$

❹ Return x

Finding initial feasible point: solve auxiliary problem

$$\underset{x,s}{\text{minimize}} \quad s \quad \text{s.t.} \quad g(x) \leq s1$$

Start with any $x^{(0)}$ and $s^{(0)} > \max_j g_j(x^{(0)})$.

- **Pro:** every iterate is feasible
- **Pro:** well-suited for convex programs (LP, QP, SOCP)
- **Con:** needs strictly feasible start
- **Con:** barrier becomes ill-conditioned as $\rho \rightarrow \infty$

Feasibility subproblem

minimize s s.t. $g(x) \leq s1$

Feasible pair (x, s) with $s < 0$

implies x is interior to the original feasible set.

Python: Interior Point Method

```
import numpy as np
from scipy.optimize import minimize

def interior_point_method(f, g_list, x0, rho=1.0, gamma=2.0, eps=1e-6):
    """
    f          : objective f(x)
    g_list     : list of inequality constraint functions g_j(x) <= 0
    x0         : strictly feasible initial point
    """
    x = np.array(x0, dtype=float)
    delta = np.inf
    while delta > eps:
        def penalized(xv):
            barriers = [-np.log(-g(xv)) for g in g_list]
            if any(np.isnan(b) or np.isinf(b) for b in barriers):
                return 1e20
            return f(xv) + sum(barriers) / rho
        res = minimize(penalized, x, method='Nelder-Mead',
                      options={'xatol': 1e-9, 'fatol': 1e-9, 'maxiter': 5000})
        delta = np.linalg.norm(res.x - x)
        x = res.x
        rho *= gamma
    return x

# Example: min (x-3)^2 s.t. x >= 1 (i.e., 1-x <= 0)
f = lambda x: (x[0] - 3)**2
g = [lambda x: 1.0 - x[0]]          # g(x) = 1 - x <= 0
```

Method Comparison

Method	Constraint	Feasibility	Key parameter	Note
Substitution	Equality	Exact	—	Reduces dim.
Projection	Convex set	Always feasible	Step size α	QP sub-problem
Penalty	Any	Approximate	$\rho \rightarrow \infty$	Ill-cond. for large ρ
Aug. Lagrangian	Equality	Exact at finite ρ	ρ, λ	More robust
Interior Point	Inequality	Always feasible	ρ (barrier)	Needs strict interior

When to use each

- **Simple bounds/affine:** substitution or projection
- **Black-box f :** penalty or augmented Lagrangian
- **Convex / LP / QP:** interior point (IPOPT, CVXPY)

SciPy interface

`scipy.optimize.minimize` supports:
`method='SLSQP'` — sequential QP
`method='trust-constr'` — trust-region
Both accept `constraints=` dicts for h, g .

Python: scipy.optimize with Constraints

```
import numpy as np
from scipy.optimize import minimize

# Minimize  $f(x) = (x_1-1)^2 + (x_2-2.5)^2$ 
# s.t.  $h(x) = x_1 + x_2 - 3 = 0$  (equality)
#  $g(x) = x_1 - x_2 + 1 \leq 0$  (inequality)

f = lambda x: (x[0]-1)**2 + (x[1]-2.5)**2

constraints = [
    {'type': 'eq', 'fun': lambda x: x[0] + x[1] - 3.0},      #  $h(x) = 0$ 
    {'type': 'ineq', 'fun': lambda x: -(x[0] - x[1] + 1.0)}, #  $-g(x) \geq 0$ 
]
bounds = [(0, None), (0, None)] #  $x_1, x_2 \geq 0$ 

res = minimize(f, [0.5, 2.0], method='SLSQP',
               constraints=constraints, bounds=bounds)
print(f"x* = {res.x}")
print(f"Equality constraint: {res.x[0]+res.x[1]-3:.2e}")
# x* ~ [1.4, 1.7], equality satisfied to machine precision
```

Listing 8: Using scipy.optimize.minimize with equality and inequality constraints

Chapter 10 — Summary

Key concepts

- Constraints restrict design space to feasible set $\mathcal{X}$
- Two fundamental types: **equality** $h(x) = 0$ and **inequality** $g(x) \leq 0$
- Variable **transformations** eliminate constraints entirely
- **LQ decomposition** removes affine equality constraints, reducing dimension
- **Projected gradient** always stays feasible; projection is a QP
- **Lagrange multipliers**: $\nabla f = \lambda \nabla h$ at equality-constrained optima
- **KKT conditions**: first-order necessary conditions for constrained optima (equality + inequality)

Practical methods

- **Slack variables** convert inequalities to equalities ($s = t^2$)
- **Penalty methods**: add $\rho p(x)$; increase ρ for feasibility, but condition worsens
- **Augmented Lagrangian** (method of multipliers): exact feasibility at finite ρ by updating λ
- **Interior point**: log/inverse barrier keeps iterates strictly feasible; $\rho \rightarrow \infty$ pushes to boundary

Key formula

Augmented Lagrangian update:
$$\lambda^{(k+1)} = \lambda^{(k)} + \rho h(x^{(k+1)})$$

Selected Exercises

Exercise 10.1

Solve $\min_x x$ subject to $x \geq 0$.

Solution: $x^* = 0$ (boundary of feasible set).

Exercise 10.5

What is the advantage of the method of multipliers over the quadratic penalty method?

Solution: It does *not* require $\rho \rightarrow \infty$ to achieve a feasible solution; the multiplier update $\lambda^{(k+1)} = \lambda^{(k)} + \rho h(x)$ drives the constraint to zero at finite ρ .

Exercise 10.6

When would you use the barrier (interior point) method instead of the penalty method?

Solution: When all iterates must remain feasible — e.g., f or constraints are undefined outside the feasible region.

Exercise 10.12

Minimize $x_1^3 + x_2^2 + x_3$ s.t. $x_1 + 2x_2 + 3x_3 = 6$.

Solution: By the method of Lagrange multipliers, the optimal solution is $x_1 = 2, x_2 = 1, x_3 = 0$.

Selected Exercises (continued)

Exercise 10.11

Give a quadratic objective with two variables where adding a linear constraint changes the optimum.

Solution: $\min x_1^2 + x_2^2$ subject to $x_1 \geq 1$.

Unconstrained: $x^* = (0, 0)$. Constrained: $x^* = (1, 0)$.

Exercise 10.7

Find a smooth problem where, for any $\rho > 0$, there exists a starting point from which steepest descent diverges.

Solution: $\min x^3$ s.t. $x \geq 0$.

Penalized: $\min_x x^3 + \rho(\min(x, 0))^2$.

For $x \ll 0$: $x^3 \approx x^3$ dominates and drives $x \rightarrow -\infty$.

Exercise 10.16

Why can Lagrange multipliers fail for $\min f(x)$ s.t. $c^2(x) = 0$?

Solution: The constraint gradient is $\nabla[c^2] = 2c\nabla c = 0$ at all feasible points, so $\nabla f = \lambda \cdot 0$ gives no information on λ . Use the equivalent constraint $c(x) = 0$ instead.

End of Chapter 10

Constraints

Algorithms for Optimization, 2nd ed.
Kochenderfer & Wheeler (2026)